Kniflash - 一款类幸存者游戏

项目简介

Kniflash 是一款基于 Qt 开发的 2D 俯视角射击游戏，玩家需要控制角色在地图上移动、拾取道具、躲避和攻击敌人，目标是尽可能长时间地生存下去。

文件结构

```
Kniflash/
├── Kniflash.pro        # qmake 项目文件，管理项目编译配置
├── main.cpp            # 主函数入口
├── mainwindow.h        # 主窗口头文件
├── mainwindow.cpp      # 主窗口实现文件
├── gameview.h          # 游戏视图头文件
├── gameview.cpp        # 游戏视图实现文件
├── gamescene.h         # 游戏场景头文件
├── gamescene.cpp       # 游戏场景实现文件
├── character.h         # 角色基类头文件
├── character.cpp       # 角色基类实现文件
├── player.h            # 玩家类头文件
├── player.cpp          # 玩家类实现文件
├── enemy.h             # 敌人头文件
├── enemy.cpp           # 敌人实现文件
├── item.h              # 道具头文件
├── item.cpp            # 道具实现文件
├── res.qrc             # Qt 资源文件，管理图片等资源
├── images/             # 存放游戏图片资源
│   ├── knife.png
│   ├── heart.png
│   ├── boots.png
│   └── splash.png
├── debug/              # 调试模式编译输出目录
├── release/            # 发布模式编译输出目录
└── README.md           # 本文档
```

文件夹说明

- `Kniflash/`：项目根目录。
- `Kniflash/images/`：存放游戏所需的图片资源。
- `Kniflash/debug/`：存放 `qmake` 在调试模式下编译生成的中间文件和可执行文件。
- `Kniflash/release/`：存放 `qmake` 在发布模式下编译生成的中间文件和可执行文件。

类和数据结构

1. `Character` (角色基类) - `character.h`，`character.cpp`

作为玩家 `Player` 和敌人 `Enemy` 的基类，继承自 `QGraphicsObject`。

主要成员变量:

- `m_health`：`int` - 当前生命值。
- `m_maxHealth`：`int` - 最大生命值。
- `m_knives`：`int` - 当前拥有的飞刀数量。
- `m_speed`：`qreal` - 移动速度。
- `m_knifeRadius`：`qreal` - 飞刀环绕半径。
- `m_rotationAngle`：`qreal` - 飞刀环绕当前角度。
- `m_rotationSpeed`：`qreal` - 飞刀环绕速度。
- `m_knifeItems`：`QList<QGraphicsPixmapItem*>` - 存储代表飞刀的 `QGraphicsPixmapItem` 对象列表。
- `m_animationTimer`：`QTimer*` - 用于控制角色动画的计时器。
- `m_knifeRotationTimer`：`QTimer*` - 用于控制飞刀旋转的计时器。
- `m_currentFrame`：`int` - 当前动画帧。
- `m_spriteSheet`：`QPixmap` - 角色的精灵图 (暂未使用，当前使用颜色代替)。
- `m_isInvincible`：`bool` - 是否处于无敌状态。
- `m_invincibilityTimer`：`QTimer*` - 无敌状态计时器。
- `m_effect`：`QGraphicsColorizeEffect*` - 用于实现受伤/无敌时的颜色效果。

主要方法:

- `Character(QGraphicsItem *parent = nullptr)`：构造函数。
- `~Character()`：析构函数。

- `boundingRect() const override`：返回对象的包围盒。
- `paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget = nullptr) ove`：绘制角色。
- `getHealth() const`：获取当前生命值。
- `setHealth(int health)`：设置当前生命值，并根据情况触发死亡。
- `getKnives() const`：获取飞刀数量。
- `addKnives(int count)`：增加飞刀数量并更新飞刀显示。
- `getSpeed() const`：获取移动速度。
- `setSpeed(qreal speed)`：设置移动速度。
- `isAlive() const`：判断角色是否存活。
- `startInvincibility(int duration)`：开始无敌状态。
- `createKnives()`：创建围绕角色的飞刀图形。
- `updateKnives()`：更新飞刀数量和环绕半径。
- `updateKnivesPosition()`：更新飞刀的旋转位置。
- `takeDamage(int damage)`：(纯虚函数) 角色受到伤害的逻辑。
- `die()`：(纯虚函数) 角色死亡的逻辑。
- `move(const QPointF& direction)`：(纯虚函数) 角色移动的逻辑。
- `attack()`：(纯虚函数) 角色攻击的逻辑。

动画与视觉:

- 通过 `m_animationTimer` 和 `paint` 方法实现简单的帧动画 (当前为颜色闪烁)。
- 通过 `m_knifeRotationTimer`，`createKnives`，`updateKnives`，`updateKnivesPosition` 实现飞刀的创建、更新数量、半径和旋转动画。
- 受伤时通过 `QGraphicsColorizeEffect` 显示红色，无敌时显示半透明。

2. `Player` (玩家类) - `player.h`，`player.cpp`

继承自 `Character`，代表玩家控制的角色。

主要成员变量:

- `m_moveDirection`：`QPointF` - 当前移动方向。
- `m_isMoving`：`bool` - 玩家是否正在移动。
- `m_targetEnemy`：`Enemy*` - 当前锁定的攻击目标。
- `m_targetingRadius`：`qreal` - 自动索敌半径。

- `m_attackCooldownTimer`: `QTimer*` - 攻击冷却计时器。
- `m_canAttack`: `bool` - 是否可以攻击。
- `m_isSpeedBoosted`: `bool` - 是否处于速度提升状态。
- `m_speedBoostTimer`: `QTimer*` - 速度提升持续时间计时器。

主要方法:

- `Player(QGraphicsItem *parent = nullptr)`: 构造函数。
- `paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget = nullptr) ove`: 绘制玩家 (当前为蓝色矩形)。
- `advance(int phase) override`: 每帧更新，处理移动等。
- `keyPressEvent(QKeyEvent *event) override`: 处理按键按下事件，控制移动和攻击。
- `keyReleaseEvent(QKeyEvent *event) override`: 处理按键释放事件，停止移动。
- `move(const QPointF& direction) override`: 实现玩家的移动逻辑，会检查边界。
- `attack() override`: 实现玩家的攻击逻辑，向锁定的敌人发射飞刀。
- `takeDamage(int damage) override`: 实现玩家受到伤害的逻辑。
- `die() override`: 实现玩家死亡的逻辑。
- `collectItem(Item *item)`: 拾取道具并应用效果。
- `checkForTargets()`: 检测并锁定范围内的敌人。
- `startSpeedBoost(qreal factor, int duration)`: 开始速度提升。
- `isMoving() const`: 返回玩家是否正在移动。

核心逻辑:

- 通过键盘输入控制移动方向 (`m_moveDirection`)。
- `advance` 方法中根据 `m_moveDirection` 调用 `move` 方法。
- 自动索敌 (`checkForTargets`) 并通过 `attack` 方法向目标发射可视化飞刀，飞刀击中目标后造成伤害。
- 拾取不同道具 (`collectItem`) 会获得不同增益，例如增加飞刀、恢复生命、提升速度。

3. `Enemy` (敌人-类) - `enemy.h`, `enemy.cpp`

继承自 `Character`，代表游戏中的敌人。

主要成员变量:

- `m_player`: `Player*` - 指向玩家对象的指针，用于追踪。

- `m_attackRange`: `qreal` - 敌人的攻击范围。
- `m_movementTimer`: `QTimer*` - 控制敌人移动和攻击逻辑的计时器。
- `m_attackCooldownTimer`: `QTimer*` - 攻击冷却计时器。
- `m_canAttack`: `bool` - 是否可以攻击。

主要方法:

- `Enemy(Player* player, QGraphicsItem *parent = nullptr)`: 构造函数。
- `paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget = nullptr) ove`: 绘制敌人 (当前为绿色矩形)。
- `advance(int phase) override`: 每帧更新，AI逻辑在此触发 (通过计时器)。
- `move(const QPointF& direction) override`: 实现敌人的移动逻辑，会检查边界。
- `attack() override`: 实现敌人的攻击逻辑 (近战攻击玩家)。
- `takeDamage(int damage) override`: 实现敌人受到伤害的逻辑。
- `die() override`: 实现敌人死亡的逻辑，可能会掉落道具。
- `updateAI()`: 更新敌人的 AI 行为，包括移动和攻击决策。

AI 行为:

- 敌人会周期性地 (`m_movementTimer`) 更新其行为 (`updateAI`)。
- 如果玩家在攻击范围内 (`m_attackRange`) 且攻击未冷却，则尝试攻击玩家。
- 否则，向玩家方向移动或随机移动。

4. `Item` (道具类) - `item.h`, `item.cpp`

继承自 `QGraphicsObject`，代表游戏中可以拾取的道具。

主要成员变量:

- `m_type`: `ItemType` (enum: `Knife`, `Heart`, `Boots`) - 道具类型。
- `m_pixmap`: `QPixmap` - 道具的图片。

主要方法:

- `Item(ItemType type, QGraphicsItem *parent = nullptr)`: 构造函数。
- `boundingRect() const override`: 返回对象的包围盒。
- `paint(QPainter *painter, const QStyleOptionGraphicsItem *option, QWidget *widget = nullptr) ove`: 绘制道具。

- `type() const`：返回道具类型。

道具类型与效果:

- `Knife`：增加玩家的飞刀数量。
- `Heart`：恢复玩家的生命值。
- `Boots`：短暂提升玩家的移动速度。

5. `GameScene` (游戏场景类) - `gamescene.h`，`gamescene.cpp`

继承自 `QGraphicsScene`，管理游戏中的所有元素和逻辑。

主要成员变量:

- `m_player`：`Player*` - 玩家对象。
- `m_enemies`：`QList<Enemy*>` - 敌人列表。
- `m_items`：`QList<Item*>` - 道具列表。
- `m_bushes`：`QList<QGraphicsEllipseItem*>` - 灌木丛列表。
- `m_gameTimer`：`QTimer*` - 游戏主循环计时器，用于更新游戏状态、产生敌人/道具等。
- `m_enemySpawnTimer`：`QTimer*` - 控制敌人产生的计时器。
- `m_itemSpawnTimer`：`QTimer*` - 控制道具产生的计时器。
- `m_mapRadius`：`qreal` - 游戏地图的半径。
- `m_isGameOver`：`bool` - 游戏是否结束。
- `m_gameStartTime`：`QElapsedTimer` - 记录游戏开始时间，用于计算生存时间。
- `m_survivalTimeText`：`QGraphicsTextItem*` - 显示生存时间的文本。
- `m_playerHealthText`：`QGraphicsTextItem*` - 显示玩家血量的文本。
- `m_playerKnivesText`：`QGraphicsTextItem*` - 显示玩家飞刀数的文本。

主要方法:

- `GameScene(QObject *parent = nullptr)`：构造函数。
- `startGame()`：初始化并开始游戏。
- `gameOver()`：处理游戏结束逻辑。
- `spawnEnemy()`：在随机位置生成一个敌人。
- `spawnItem()`：在随机位置生成一个道具。
- `createMap()`：创建游戏地图边界和背景。
- `createBushes()`：在地图上随机生成灌木丛。

- `updateGame()`：游戏主更新逻辑，包括碰撞检测、UI更新等。
- `checkCollisions()`：检测玩家与敌人、玩家与道具、敌人与子弹（飞刀）之间的碰撞。
- `updateUI()`：更新游戏界面上的信息，如生存时间、血量、飞刀数。
- `getRandomSpawnPosition()`：获取地图内的一个随机有效生成位置。
- `isPositionValid(const QPointF& pos, qreal radius)`：检查给定位置是否在地图边界内且不与其他物体过于接近。

游戏流程:

1. `startGame()` 被调用，初始化玩家、地图、UI、计时器。
2. `m_gameTimer` 周期性调用 `updateGame()`。
3. `updateGame()` 中调用 `checkCollisions()` 和 `updateUI()`。
4. `m_enemySpawnTimer` 和 `m_itemSpawnTimer` 周期性调用 `spawnEnemy()` 和 `spawnItem()`。
5. 玩家通过键盘控制，敌人通过 AI 移动和攻击。
6. 当玩家生命值小于等于0时，触发 `gameOver()`。

6. `GameView` (游戏视图类) - `gameview.h`，`gameview.cpp`

继承自 `QGraphicsView`，用于显示 `GameScene`。

主要成员变量:

- `m_gameScene`：`GameScene*` - 指向当前游戏场景。
- `m_gameOverText`：`QGraphicsTextItem*` - 用于显示游戏结束信息的文本。
- `m_restartButton`：`QPushButton*` (如果使用按钮重启) 或通过事件处理。

主要方法:

- `GameView(QWidget *parent = nullptr)`：构造函数。
- `startNewGame()`：开始一局新游戏。
- `showGameOverScreen(int survivalTime)`：显示游戏结束界面和生存时间。
- `keyPressEvent(QKeyEvent *event) override`：将按键事件转发给场景中的玩家。
- `keyReleaseEvent(QKeyEvent *event) override`：将按键事件转发给场景中的玩家。
- `wheelEvent(QWheelEvent *event) override`：处理鼠标滚轮事件，用于缩放视图。
- `resizeEvent(QResizeEvent *event) override`：视图大小改变时，确保场景适应。
- `ensurePlayerVisible()`：确保玩家在视图中可见，通常使视图中心跟随玩家。

核心功能:

- 作为 `GameScene` 的容器和渲染器。
- 处理游戏开始和结束时的界面切换。
- 将用户输入（键盘、鼠标）传递给 `GameScene` 或直接处理（如缩放）。
- 确保玩家角色始终在视野中心。

7. `MainWindow` (主窗口类) - `mainwindow.h`, `mainwindow.cpp`

继承自 `QMainWindow`，作为游戏的顶层窗口，包含主菜单和游戏视图。

主要成员变量:

- `m_gameView`: `GameView*` - 游戏视图对象。
- `m_mainMenuWidget`: `QWidget*` - 主菜单界面。
- `m_startGameButton`: `QPushButton*` - 开始游戏按钮。
- `m_exitButton`: `QPushButton*` - 退出游戏按钮。

主要方法:

- `MainWindow(QWidget *parent = nullptr)`: 构造函数，初始化主菜单。
- `showMainMenu()`: 显示主菜单界面。
- `startGame()`: 隐藏主菜单，创建并显示游戏视图，开始新游戏。
- `closeEvent(QCloseEvent *event) override`: 处理窗口关闭事件。

8. `main.cpp`

程序入口点。

主要功能:

- 创建 `QApplication` 对象。
- 创建并显示一个启动画面 (Splash Screen)。
- 创建并显示 `MainWindow`。
- 进入 Qt 事件循环 `app.exec()`。

编译与运行

1. 确保已安装 Qt SDK (推荐 Qt 5.15.x 或更高版本)。
2. 打开 Qt Creator，加载 `Kniflash.pro` 文件。
3. 配置项目使用合适的 Kit (例如 MinGW for Windows)。
4. 构建项目 (Build)。
5. 运行项目 (Run)。

或者通过命令行:

```
cd path/to/Kniflash
qmake
make # 或者 mingw32-make，取决于你的编译环境
./Kniflash # 或者 ./debug/Kniflash 或 ./release/Kniflash
```

游戏玩法说明

游戏目标

- **生存**: 玩家需要操控角色在布满敌人的地图上尽可能长时间地生存下去。
- **消灭敌人**: 击败敌人可以获得分数（当前未在UI显示，但逻辑中存在），清理出更安全的活动空间。
- **收集道具**: 地图上会随机刷新各种道具，拾取它们可以增强角色的能力。

操作方式

- **移动**:
  - 使用 `W`，`A`，`S`，`D` 键控制角色向上、左、下、右移动。
  - 也支持使用键盘的 `上箭头`，`左箭头`，`下箭头`，`右箭头` 进行移动。
- **攻击**:
  - 按 `空格键` 进行攻击。
  - 玩家会自动锁定攻击范围内最近的敌人。
  - 攻击会消耗一把环绕在角色身边的飞刀，飞刀会自动飞向锁定的目标。
- **退出游戏/返回主菜单**:
  - 在游戏过程中，按 `ESC` 键可以立即退出当前游戏并返回主菜单。

核心机制

- **飞刀系统**:
  - 角色初始拥有4把飞刀环绕自身。
  - 当飞刀数量少于4把时，每秒会自动补充一把。
  - 飞刀数量越多，飞刀的环绕半径越大，同时玩家的自动索敌范围也会相应增大。
  - 远程攻击会消耗飞刀。
  - 近战：当玩家与敌人近距离接触时，如果双方都有飞刀，会相互抵消飞刀；如果一方有飞刀而另一方没有，有飞刀的一方会对无飞刀的一方造成伤害并消耗一把飞刀。
- **道具系统**:
  - **飞刀 (Knife)**: 拾取后增加一把飞刀。
  - **治愈之心 (Heart)**: 拾取后恢复玩家一定量的生命值。
  - **疾步之靴 (Boots)**: 拾取后在一定时间内提升玩家的移动速度。
- **敌人 (Enemy)**:
  - 敌人会在地图上随机游走。
  - 当玩家进入其索敌范围后，敌人会锁定玩家并发起远程攻击。
  - 敌人被消灭后有几率掉落道具（当前未实现，但死亡逻辑已包含）。
- **地图与视野**:
  - 游戏地图是一个广阔的圆形区域，有不可穿越的边界。
  - 游戏视角会始终跟随玩家角色。
  - 地图上随机分布着一些灌木丛，角色进入灌木丛会被部分遮挡（视觉上，灌木丛的Z值较高）。
- **游戏结束**:
  - 当玩家生命值降为0时，游戏结束。
  - 游戏结束后会显示"失败！"字样，以及本局的击杀数和生存时间，并提供返回主菜单的按钮。
  - 如果玩家成功消灭地图上所有敌人（当前敌人数量固定），则游戏胜利，显示"胜利!"。